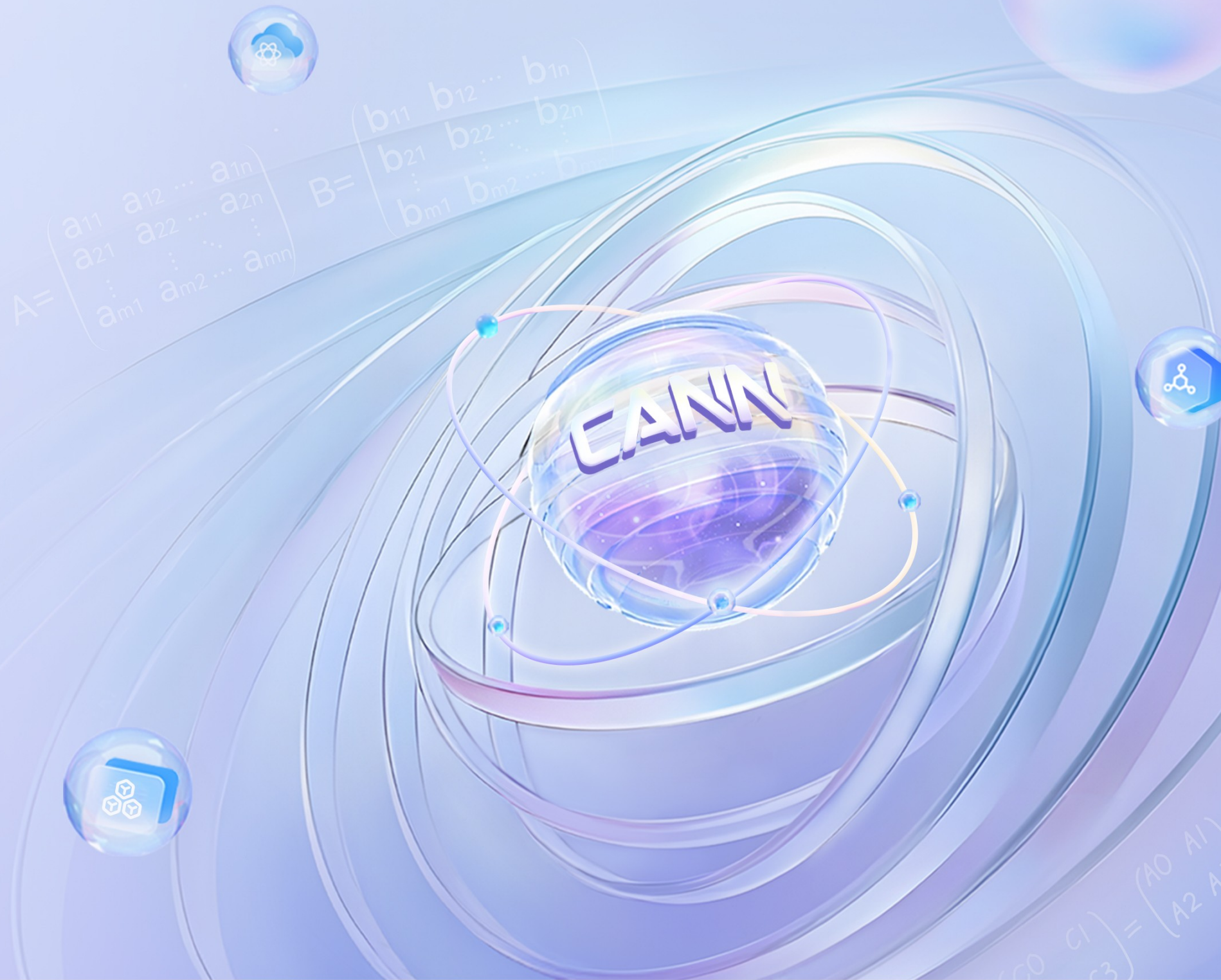


大语言模型基础 从文本到推理系统

昇腾 CANN 启航营

大模型推理基础



目录

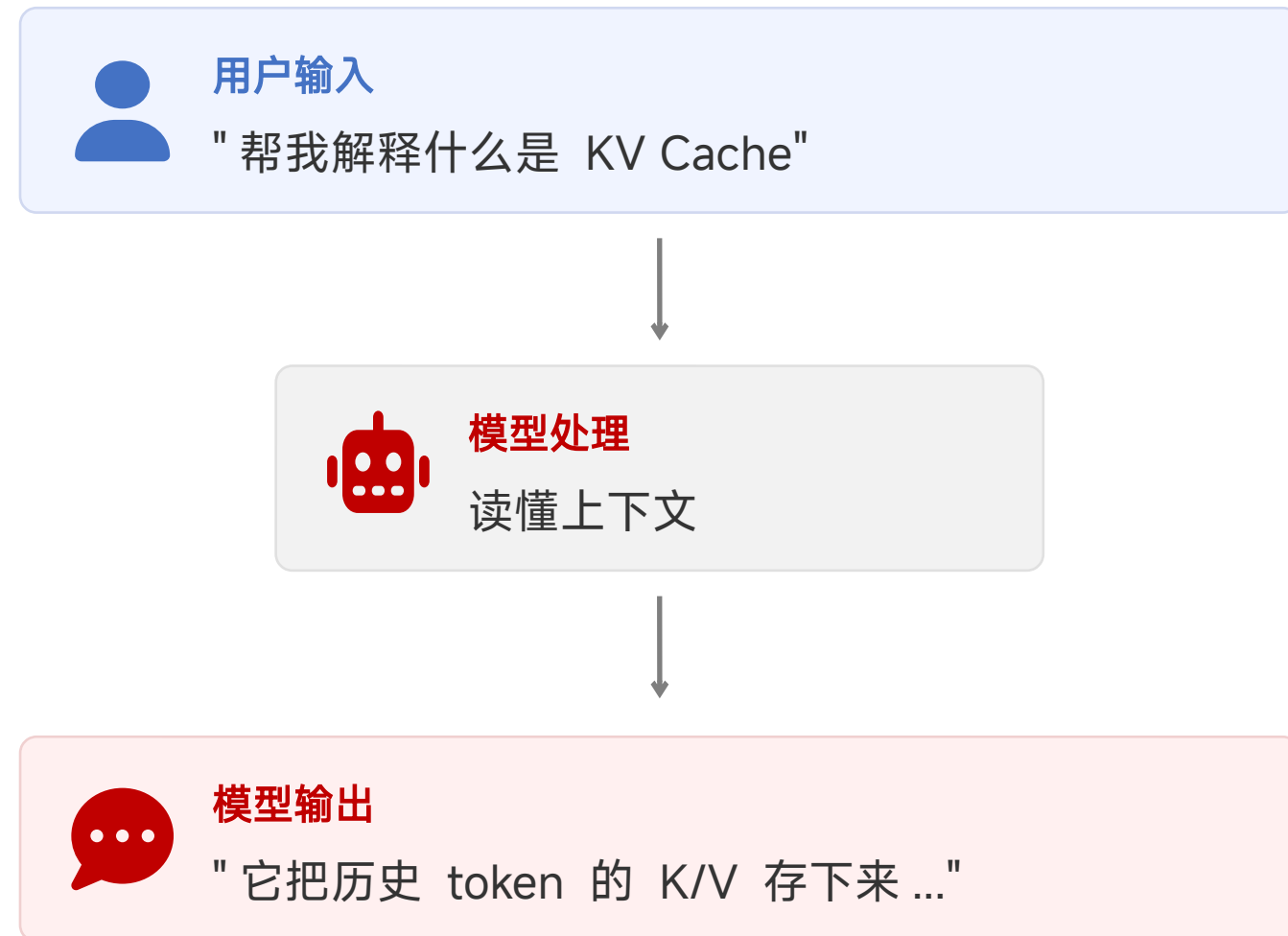
- 1 文本如何进入模型
- 2 模型如何生成回答
- 3 Transformer 与 Qwen 类结构
- 4 大模型推理流程
- 5 从语言模型到真实应用

01 文本如何进入模型

从一句中文开始，理解文本如何转换为可计算表示



大语言模型的基本生成任务



生成过程特点

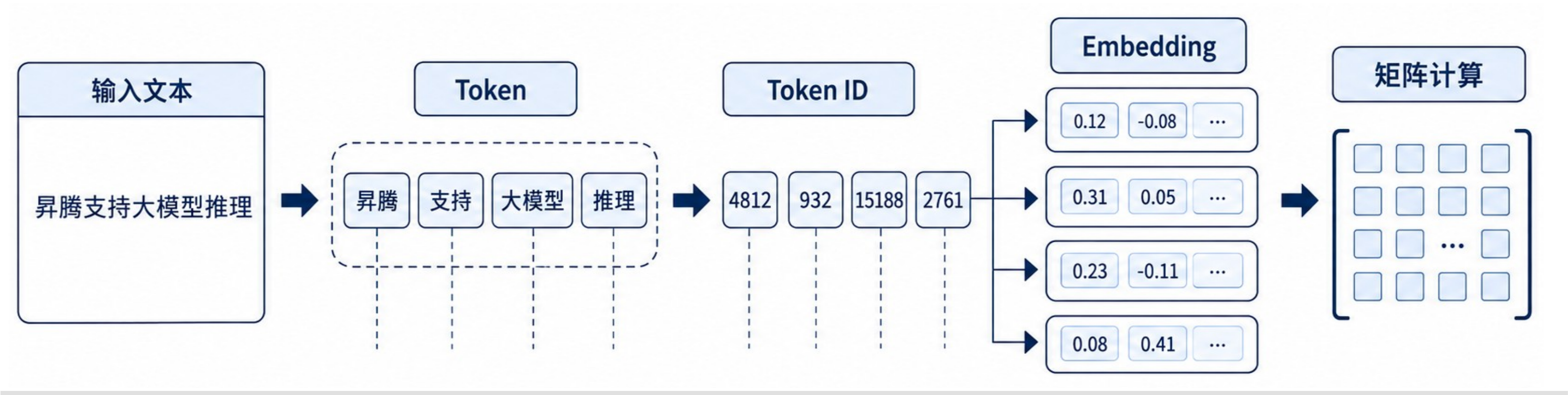
逐步生成 不是一次性拿出完整答案，而是在已有上下文后面不断选择下一个 token

连续预测 每生成一个 token，它就成为新上下文的一部分，参与下一轮预测

流式输出 对话看起来是实时输出，内部是不断循环的预测过程

大模型的文本生成过程，是在已有上下文后连续地输出后续 token。

文本需要转换为可计算表示



切分

文字 → token
把连续文本切成片段

编号

token → token id
查词表映射为编号

向量化

token id → embedding
编号变成可计算向量

核心思路：语言模型的第一步，是把语言问题变成数学计算问题。

文本切分：从字符串到 token 序列

输入文本： " 昇腾支持大模型推理 "



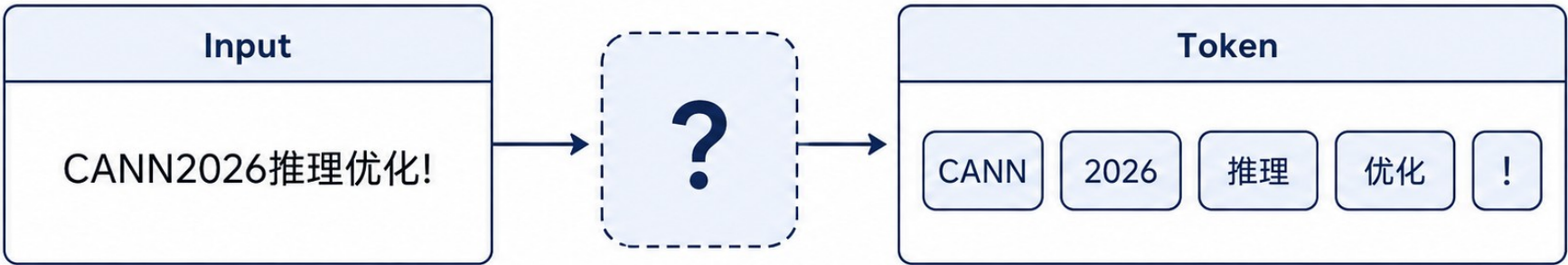
token（词元）是什么？

- token 是自然语言的基础处理单元
- 不固定等于 " 一个汉字 " 或 " 一个单词 "
- 不同 tokenizer 的切分结果可能不同
- 模型处理的是 **token 序列**

词表（ Vocab ）的作用

- 词表 = token 与编号的映射字典
- 输入被映射为编号序列
- 编号本身没有语义
- 真正参与计算的是 embedding 向量

补充： 中文、英文、数字和特殊符号都可能形成 token ； tokenizer 的作用是把输入切成模型词表里能处理的片段；不同模型的 tokenizer 可能不同。



Embedding : token id 转换为向量表示

Embedding 查表

token id → 向量

例: token id 15188 ("大模型")

→ 向量 [0.23, -0.11, 0.07, ...]

按 id 查表, 每个 token 对应一个固定维度的向量

位置信息: token 顺序影响语义

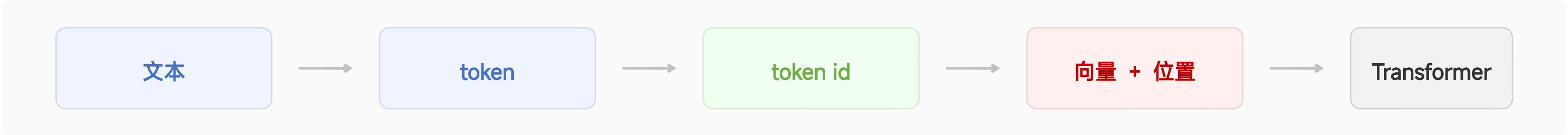
句子 A: 模型 优化 推理

pos 1 → pos 2 → pos 3

句子 B: 推理 优化 模型

相同 token, 顺序不同, 语义完全不同

模型输入 = 内容 + 位置

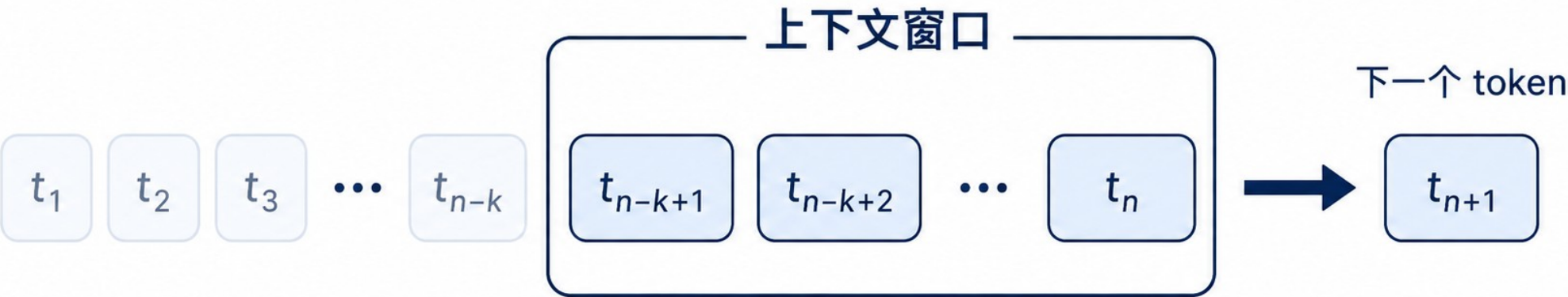


向量让模型能用数学计算处理语言含义, 而位置信息让模型知道每个 token 的顺序, 从而区分词序带来的语义差异。

上下文窗口与模型输入总结

上下文窗口 (Context Window)

上下文窗口表示模型在本轮预测下一个 token 时可参考的 token 范围。本课程中，上下文窗口一般指全部历史文本
窗口更长 → 能读更多历史和更长文档，模型记忆力更好 | **代价更高** → 计算和内存压力上升



从文本到 Transformer 输入



小结： Transformer 层处理的对象，就是这些带位置信息的向量。

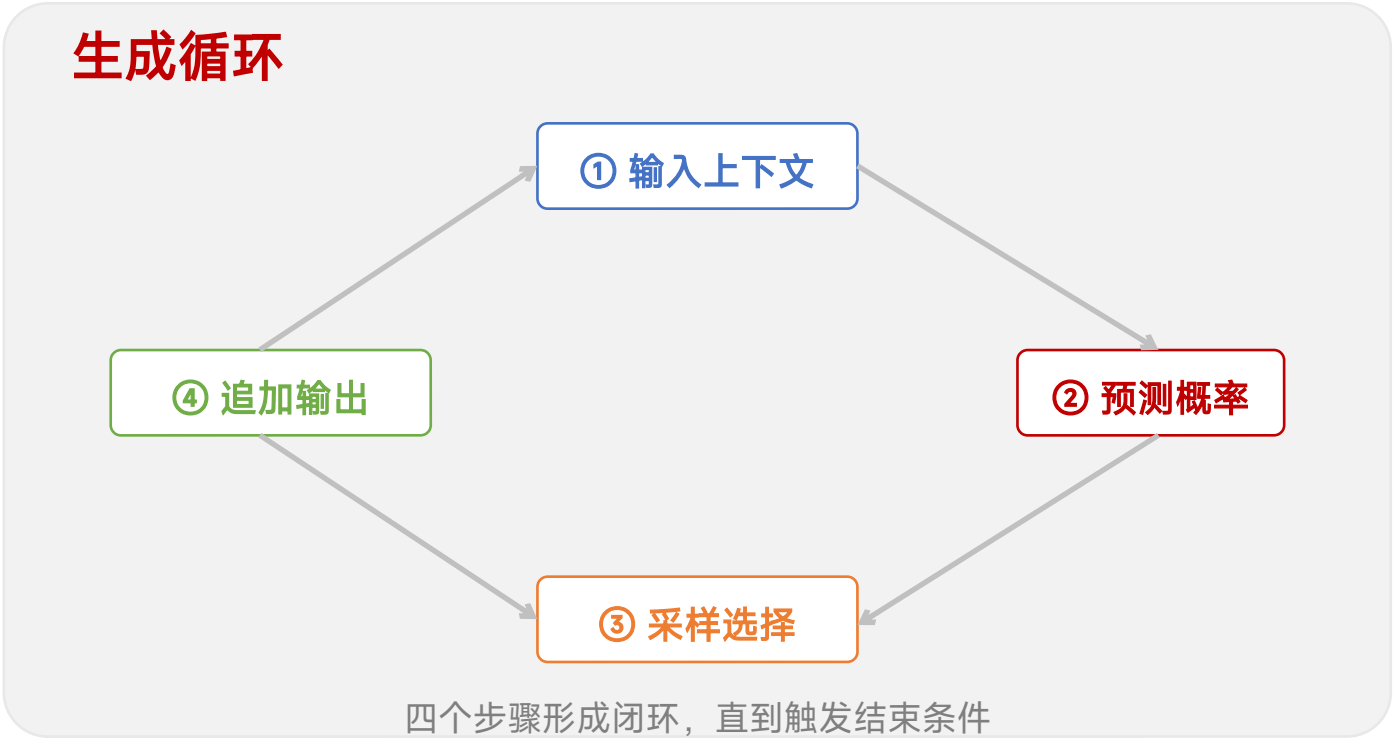
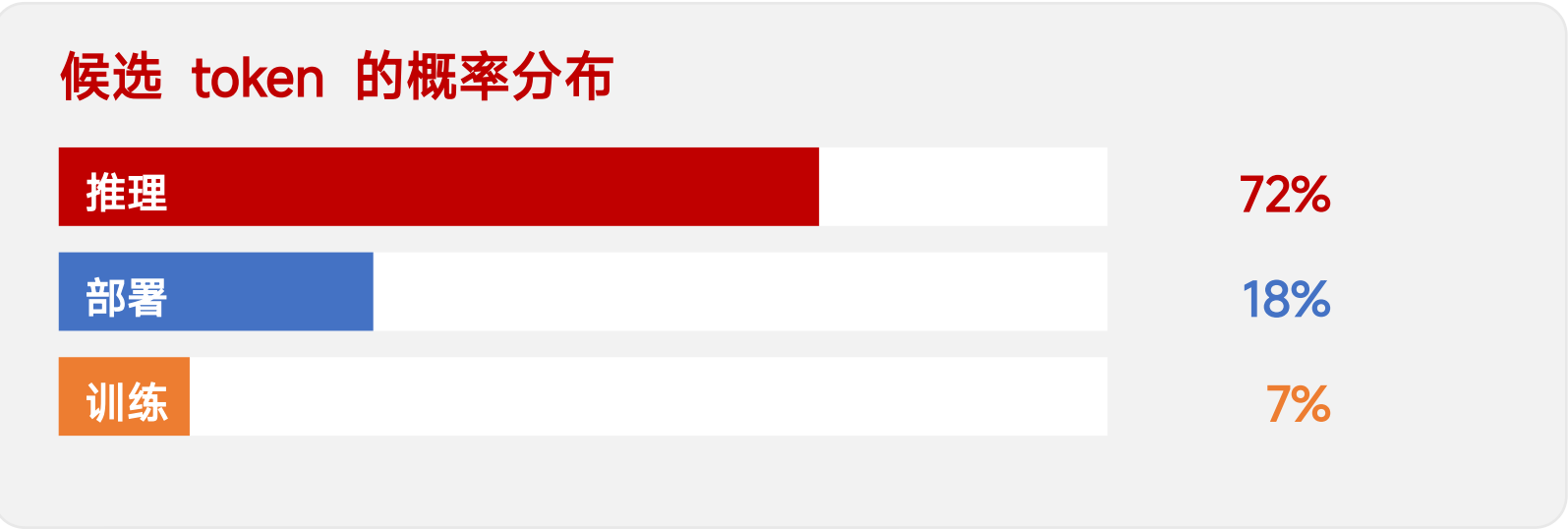
02 模型如何生成回答

预测下一个 token，是生成回答的基本机制



自回归生成：反复预测下一个 token

Prompt : "昇腾 CANN 可以帮助大模型 ____"



大语言模型的生成过程，本质上就是反复预测下一个 token 。模型不是只输出一个固定答案，而是先给出多个候选 token 的概率分布，再从中选择一个。

关键理解：每次生成新 token 后，它会立即被追加到输入上下文中，成为下一步预测的输入——这就是 "自回归" 的核心机制。

采样策略与回答风格



更保守

总是选择概率最高的 token
输出稳定，但可能单调重复
适合：代码生成、事实回答

temperature ≈ 0



适度随机

在高概率候选中选择
回答更自然、有变化
适合：对话、创意写作

temperature ≈ 0.7



过度随机

低概率 token 也可能被选中
输出容易跑偏、不连贯
适合：头脑风暴、探索

temperature > 1.0

温度 (temperature) 和 top-p 等参数，本质上是在控制 "稳定" 和 "多样" 的平衡。不同任务适合不同的参数设置，比如代码生成适合低温度，创意写作适合高温度。

生成循环与 Prompt 的约束作用

生成循环机制



Prompt 的约束作用

含糊输入

"介绍一下推理。" → 模型不知道是指数学推理、模型推理还是工程部署

清晰输入

"用 3 句话解释大模型推理中的 prefill 和 decode。" → 上下文更明确，输出更可控

Prompt Engineering 的核心，是给模型提供足够清楚的上下文和约束。好的 Prompt 能让模型输出更准确、更有用。

大模型的能力边界

✓ 模型擅长

- 根据上下文生成连贯、流畅的文本
- 理解指令并按格式输出
- 在训练数据分布内的问题表现良好

! 模型局限

- 可能编造事实或误解问题（幻觉）
- 流畅输出不代表事实正确
- 无法访问训练数据之外的信息

如何让模型更可靠？



重要结论需要工具、数据或人工确认。工具调用、检索增强和 Agent 等技术，都是为了让模型更可靠地完成任务。

03 Transformer 与 Qwen 类结构

从 Transformer Block 出发，理解 Attention、MLP 与整网路径

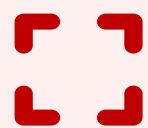
模型能力的三类规模化来源



数据规模

从大量文本、代码和多模态数据中学习模式

数据质量决定模型上限，数据量决定模型能否达到上限



参数规模

更多可学习权重，承载更复杂的语言关系

参数量决定模型的表达能力，影响推理时的计算量



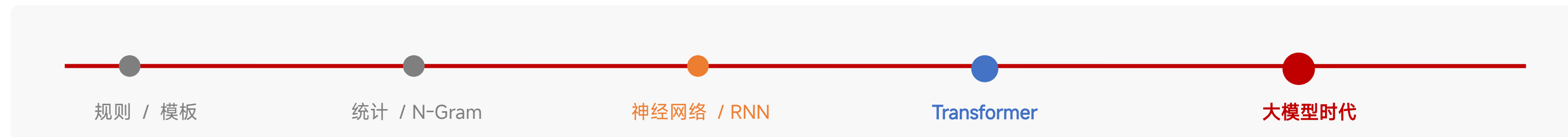
计算规模

更大训练算力，使模型充分学习数据规律

训练计算量影响模型能否充分利用数据和参数

三类规模化**共同作用**，推动模型从语言补全走向对话、推理和工具使用。三者缺一不可，共同决定了模型的能力上限。

语言模型的发展脉络



主线变化

发展趋势：人工设计特征越来越少，模型从数据中学习表示的能力越来越强。

训练与推理的职责差异

训练 Training

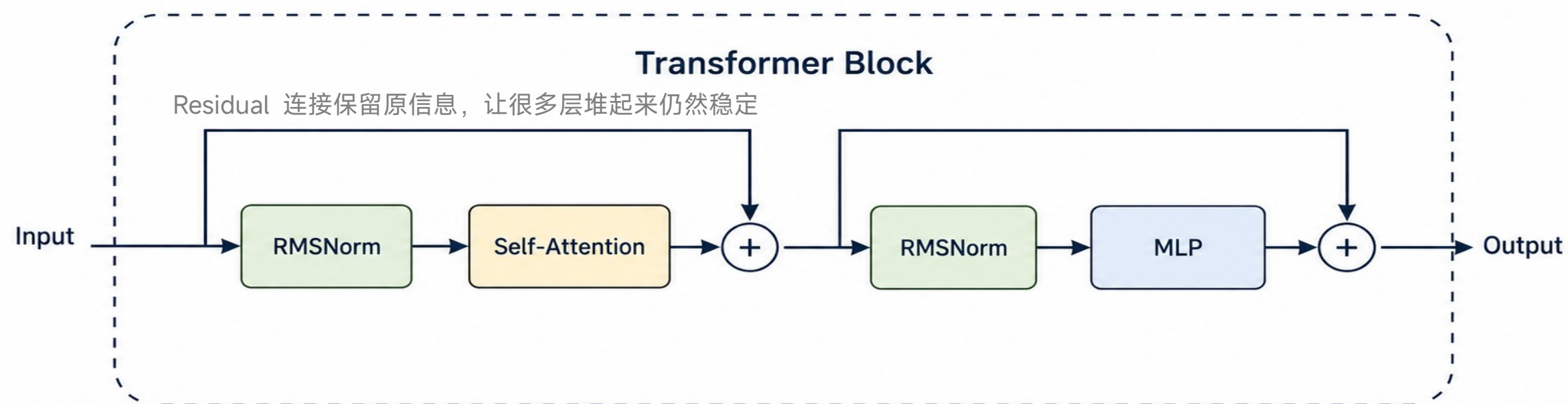
- 用途：让模型获得能力
- 计算：前向、反向、参数更新
- 关注：收敛效果、训练吞吐

推理 Inference

- 用途：使用固定模型生成回答
- 计算：前向计算与生成循环
- 关注：首 token 延迟、生成速度

Transformer 结构的核心价值

一个 Transformer Block 的核心结构



Transformer 结构的核心价值



并行处理

训练效率更高，所有 token 同时计算



看全局上下文

Attention 让 token 参考全部上下文信息



容易堆深

重复堆叠形成更强表达能力

主流大语言模型都采用 Transformer Block 反复堆叠的方式构建。

Attention 机制：从上下文中取有用信息



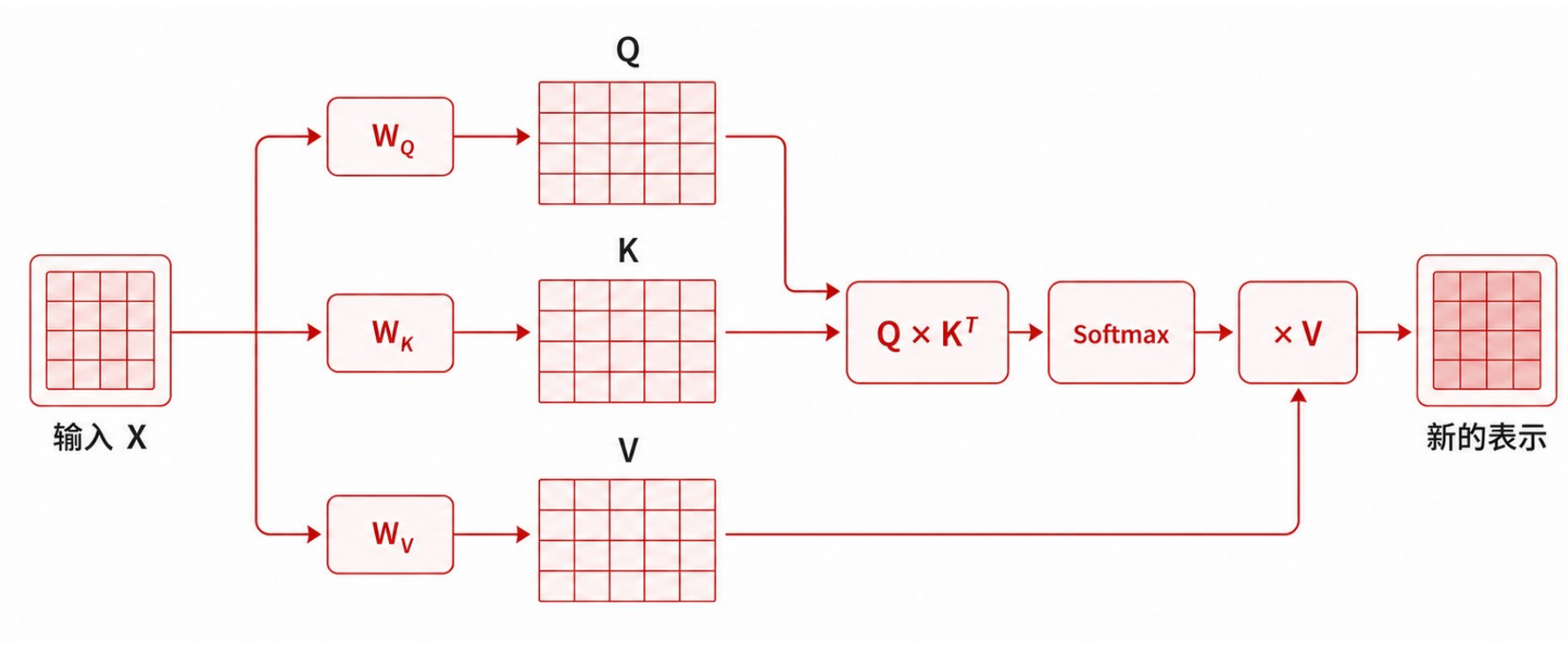
Attention 机制：当前 token 计算自己和上下文的相关性，把相关信息融合成新的表示。相关性计算是通过矩阵乘法完成的

Attention 计算：Q、K、V 与矩阵乘法

Q (Query)： 我要找什么？
—— 当前 token 的查询需求

K (Key)： 我有什么标签？
—— 上下文 token 的匹配线索

V (Value)： 我携带什么内容？
—— 上下文 token 的实际信息



Attention 公式

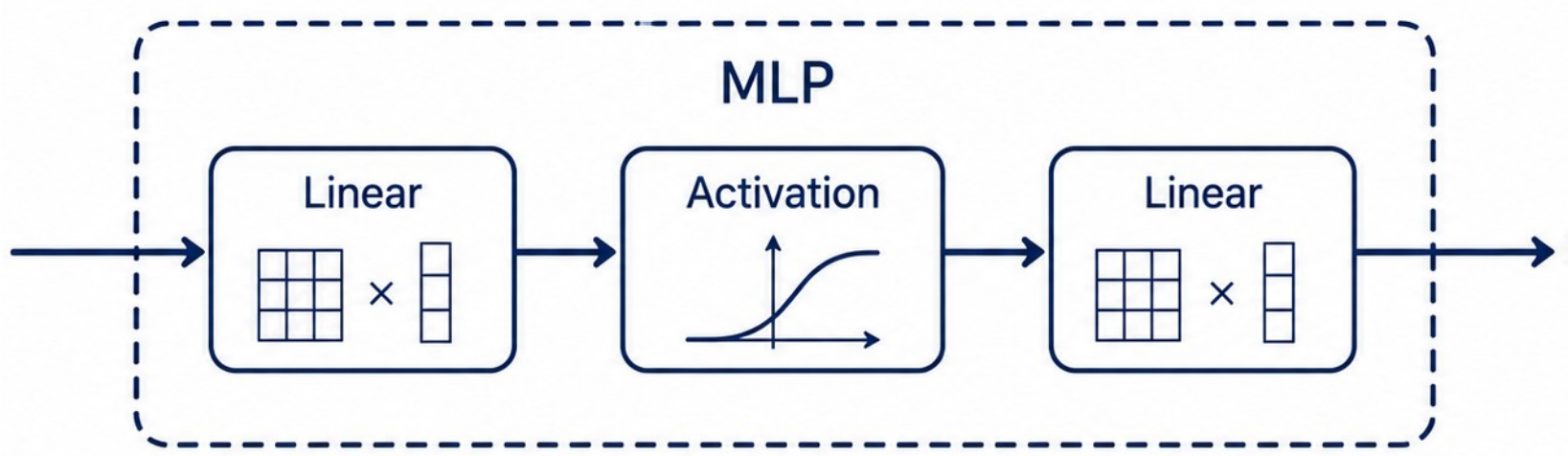
$$Attention(Q, K, V) = \text{SoftMax}\left(\frac{QK^T}{\sqrt{\quad}}\right)V$$

- QK^T ：计算每对 token 的相关性
- Softmax：把相关性变成可加的权重
- $\times V$ ：按权重汇总上下文信息

三步对应：计算相关性 → 归一化权重 → 汇总信息，得到当前 token 的新表示

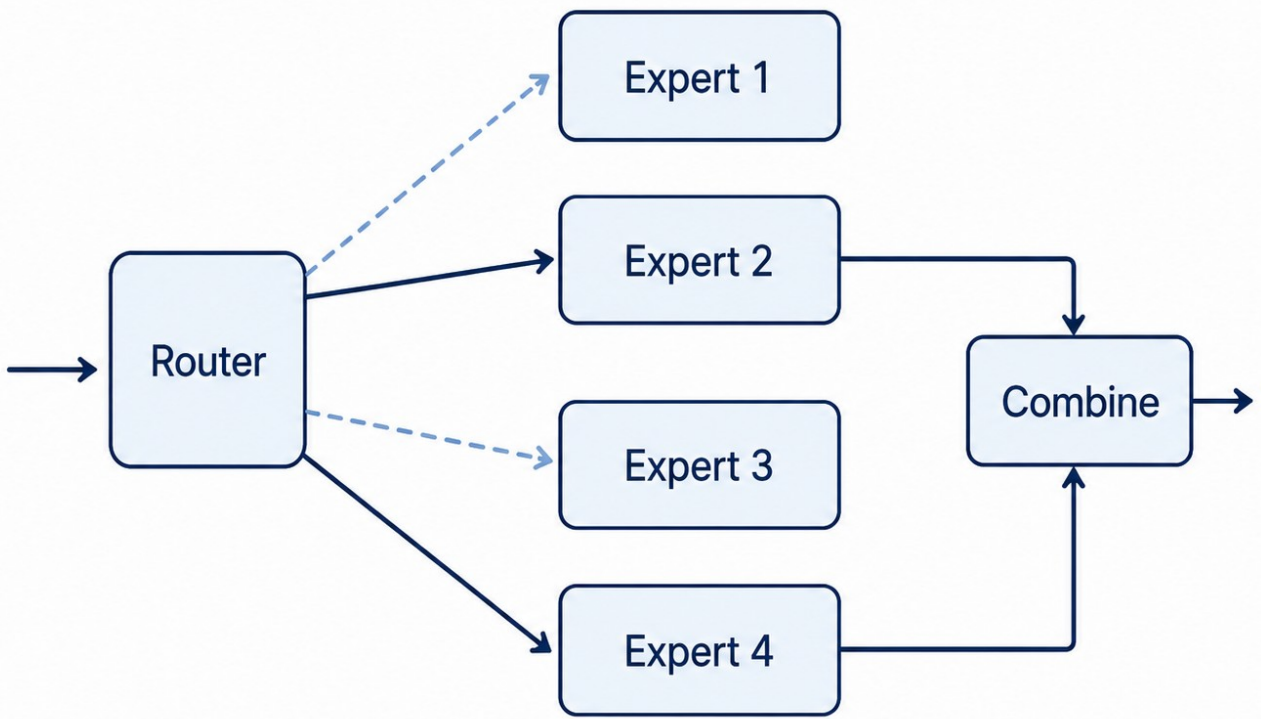
MLP 与 MoE 模型

MLP：特征变换



Attention 已经把上下文信息汇入当前 token，MLP 继续做非线性特征变换。

MoE：为 token 选择专家



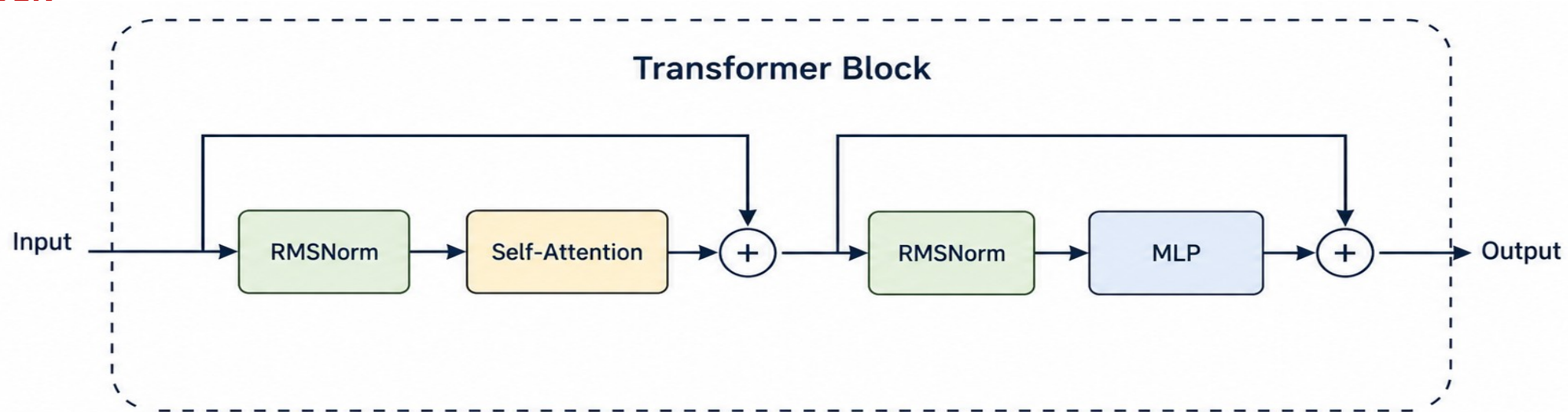
Router 为每个 token 选择 Top-K 个专家，未选中的专家不参与计算，大幅降低推理成本。

Dense vs MoE 对比

维度	Dense 模型	MoE 模型
MLP 结构	所有 token 共用同一套 MLP	准备多个 Expert MLP
计算特点	每个 token 经过完整 MLP	Router 为 token 选择少数专家

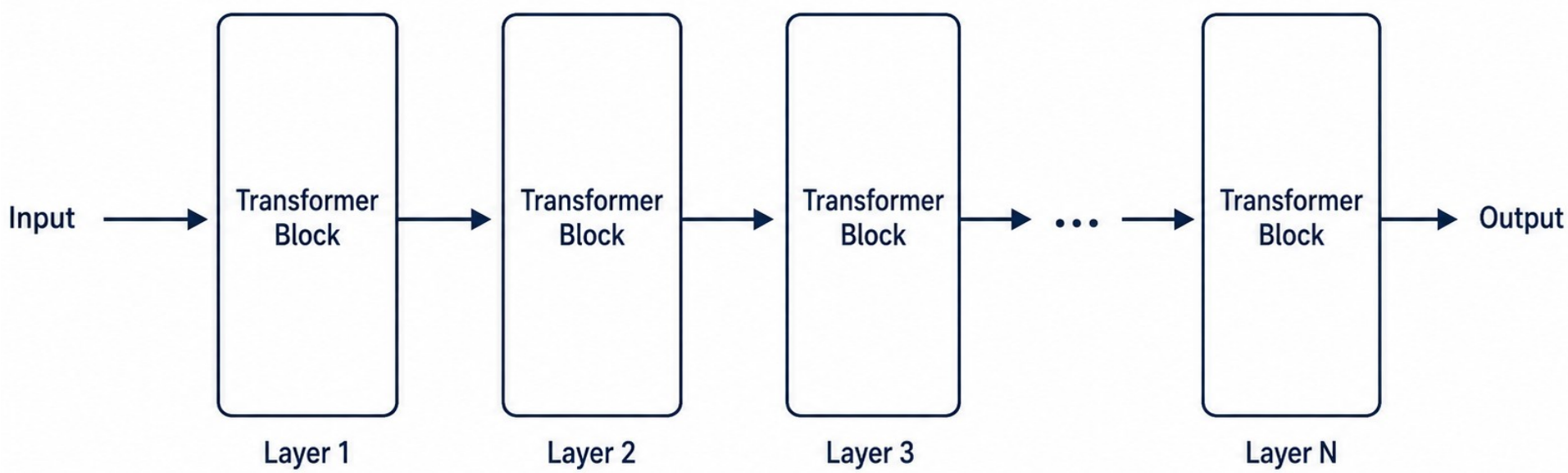
Transformer Block 主路径与多层堆叠

一个 Transformer Block



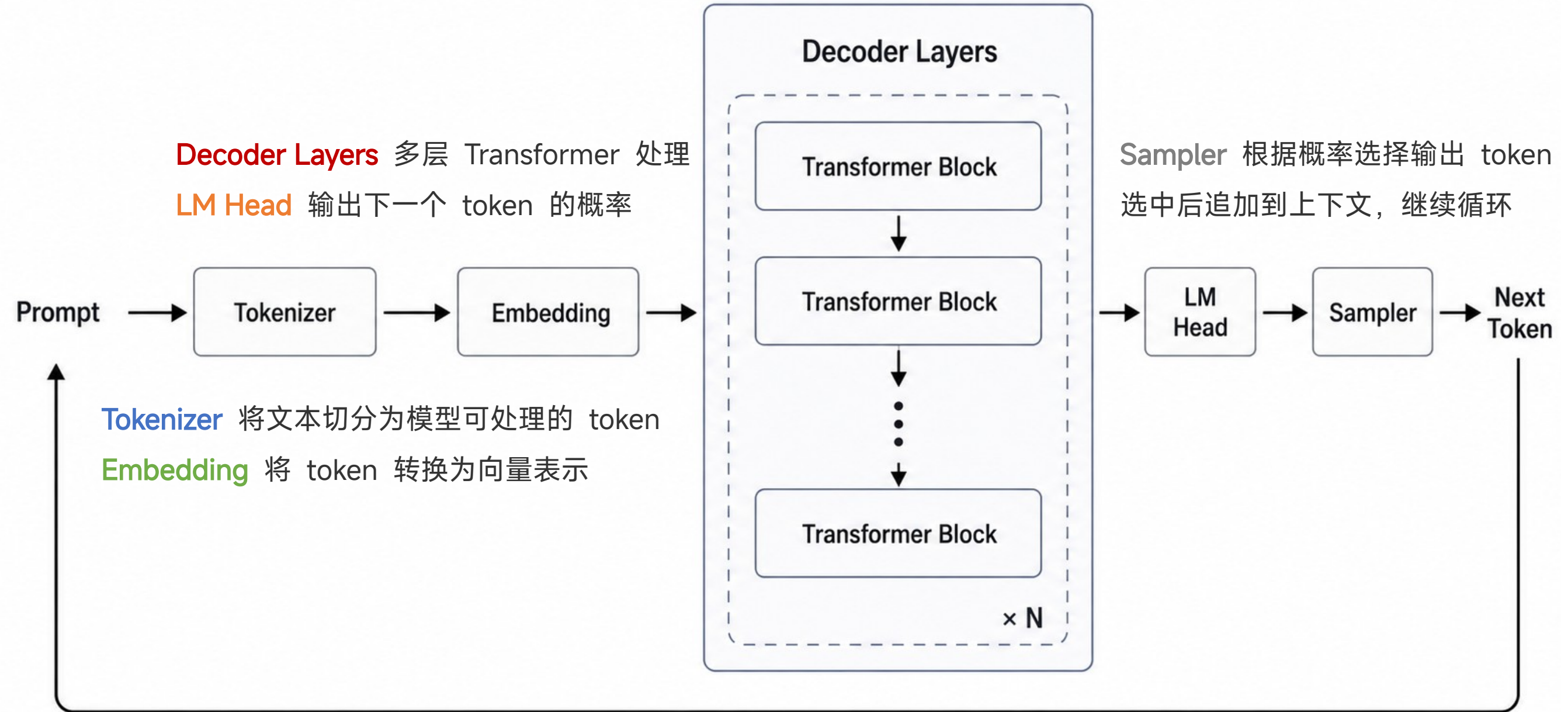
多层堆叠：从局部到全局

- 浅层：局部模式、短程关系
- 中层：语义组合、任务线索
- 深层：复杂推理、输出准备



多层堆叠让 token 表示从局部特征逐步形成更高层语义。Residual 连接保留原信息，即使堆很多层也能稳定训练。

Qwen 类模型的端到端路径



关键：反复执行这条路径，每轮生成一个 token，直到生成完整回答。

04 大模型推理流程

将 prompt、prefill、decode、KV Cache 和性能指标连接成推理时间线

文本推理路径：从 Prompt 到 Answer



Prefill 阶段

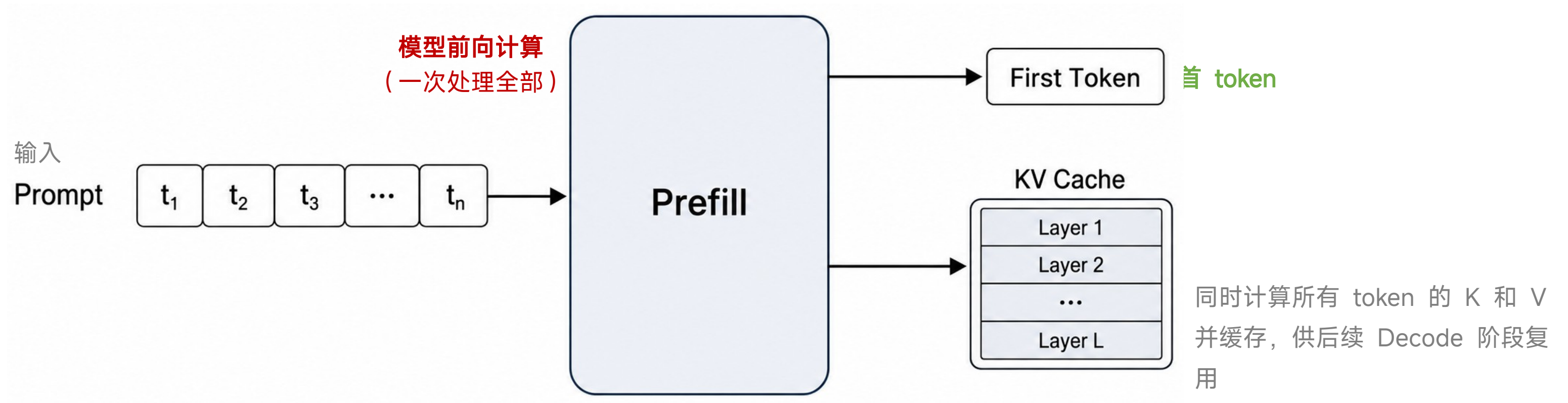
- 一次处理完整 Prompt 中的所有 token
- 决定首 token 延迟 (TTFT)
- 输入越长，一次性计算越重
- 同时计算并缓存 KV

Decode 阶段

- 每一步生成一个新 token
- 循环执行多步
- 决定持续输出速度 (TPOT/TPS)
- 对延迟和缓存读取敏感

Prefill 和 Decode 是两个差异很大的阶段：Prefill 关注“算得多快”，Decode 关注“持续生成得多快”。优化时需要分别考虑。

Prefill：一次处理完整 Prompt



Prefill 阶段特征



计算密集型

矩阵乘法为主，可充分利用并行计算



影响 TTFT

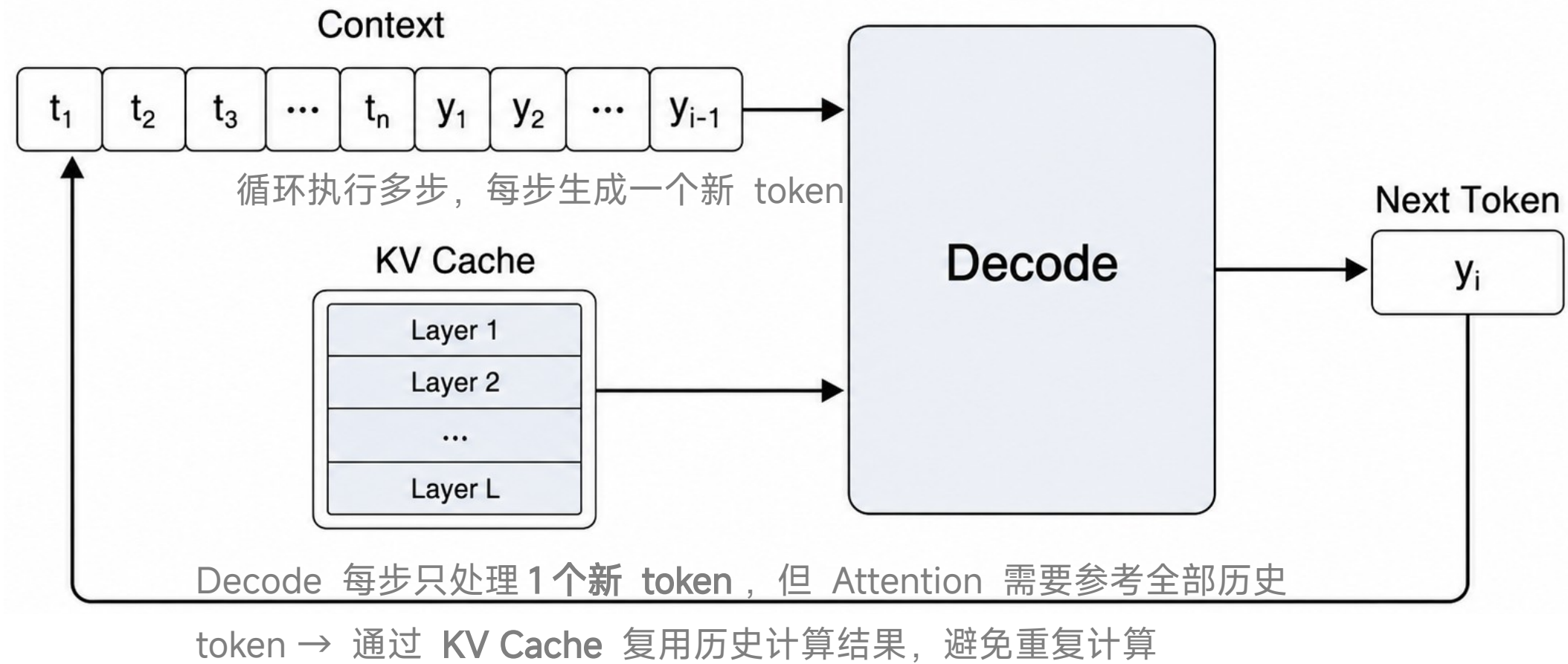
首 token 延迟 = 用户感知到的 "响应速度"



输入长度敏感

Prompt 越长，计算量越大，延迟越高

Decode : 逐步生成新 token



Decode 阶段特征

访存密集型

每步需要读取 KV Cache, 访存压力大

影响 TPOT/TPS

每 token 耗时决定用户感知的 "打字速度"

循环次数多

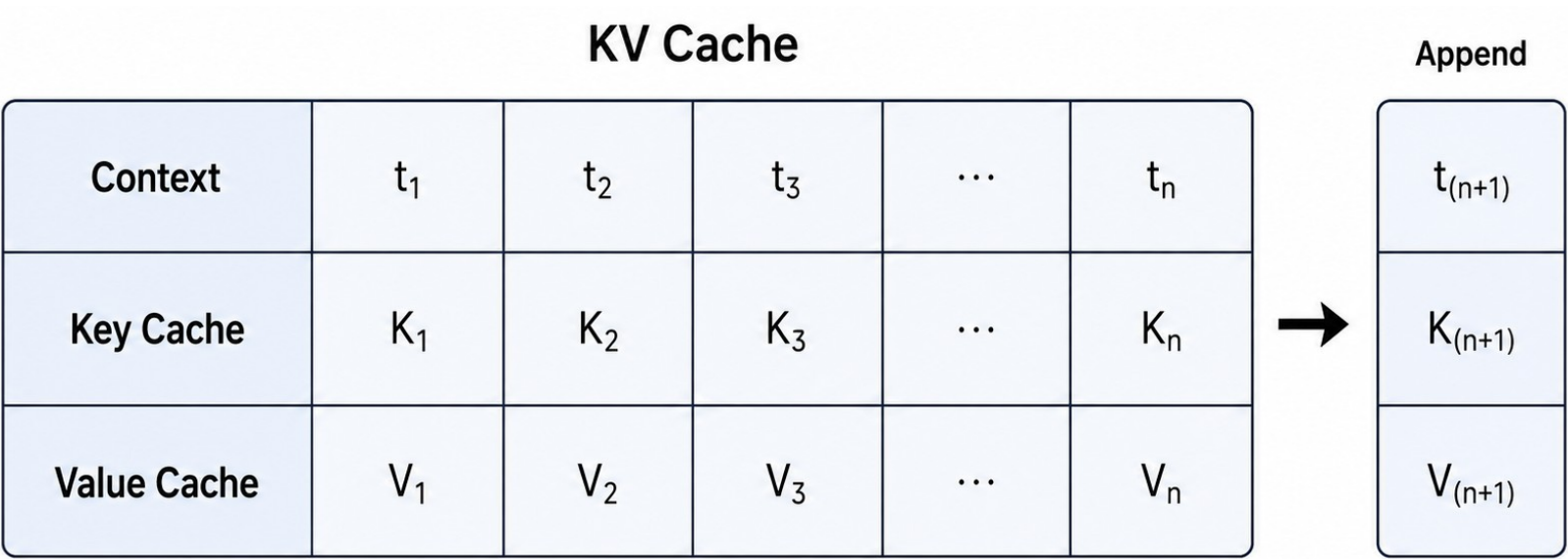
输出越长, step 越多, 对延迟敏感

KV Cache：缓存复用与显存权衡

核心思想

Decode 阶段反复使用历史 token 的 Key 和 Value。如果每次重新计算，浪费巨大。

解决方案：Prefill 阶段缓存所有历史 K/V，Decode 阶段直接读取复用。



收益

- 复用历史 K/V，降低 decode 重复计算
- 大幅减少 Attention 的计算量
- 是长文本生成的必要优化

代价

- 上下文越长，cache 越大
- 显存占用越高
- 可能成为长上下文部署的瓶颈

核心取舍：用显存换取计算速度。KV Cache 的优化也是推理系统研究的重要方向。

常见推理性能指标

TTFT

首 token 延迟

Time To First Token
用户发送请求到收到第一个 token 的时间

主要由 Prefill 阶段决定

TPOT

生成间隔

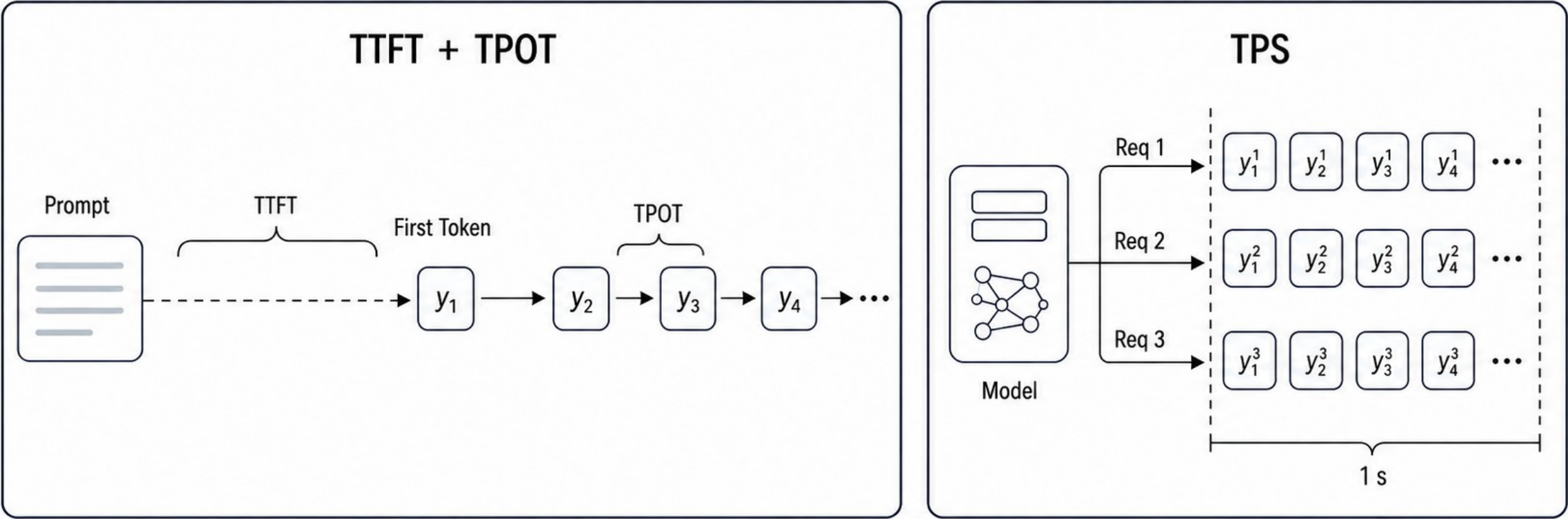
Time Per Output Token
相邻两个输出 token 之间的平均间隔
主要由 Decode 阶段决定

TPS

吞吐率

Tokens Per Second
单位时间内生成的 token 数量
综合指标，与并发相关

生成时间线示意



三个指标分别衡量首 token 等待、持续生成速度和系统吞吐能力，是评估推理性能的关键。

推理系统复杂度与优化方向

模型能力提升带来的压力

参数变大

权重占用和计算压力增加

上下文变长

KV Cache 变大，显存压力增加

结构变复杂

MoE、长上下文、量化等新特性

四大优化方向



融合算子

减少中间读写和调度开销，提升计算效率



KV Cache 优化

降低长上下文显存和访存压力



图模式

减少 Python 动态调度开销，静态优化执行



量化

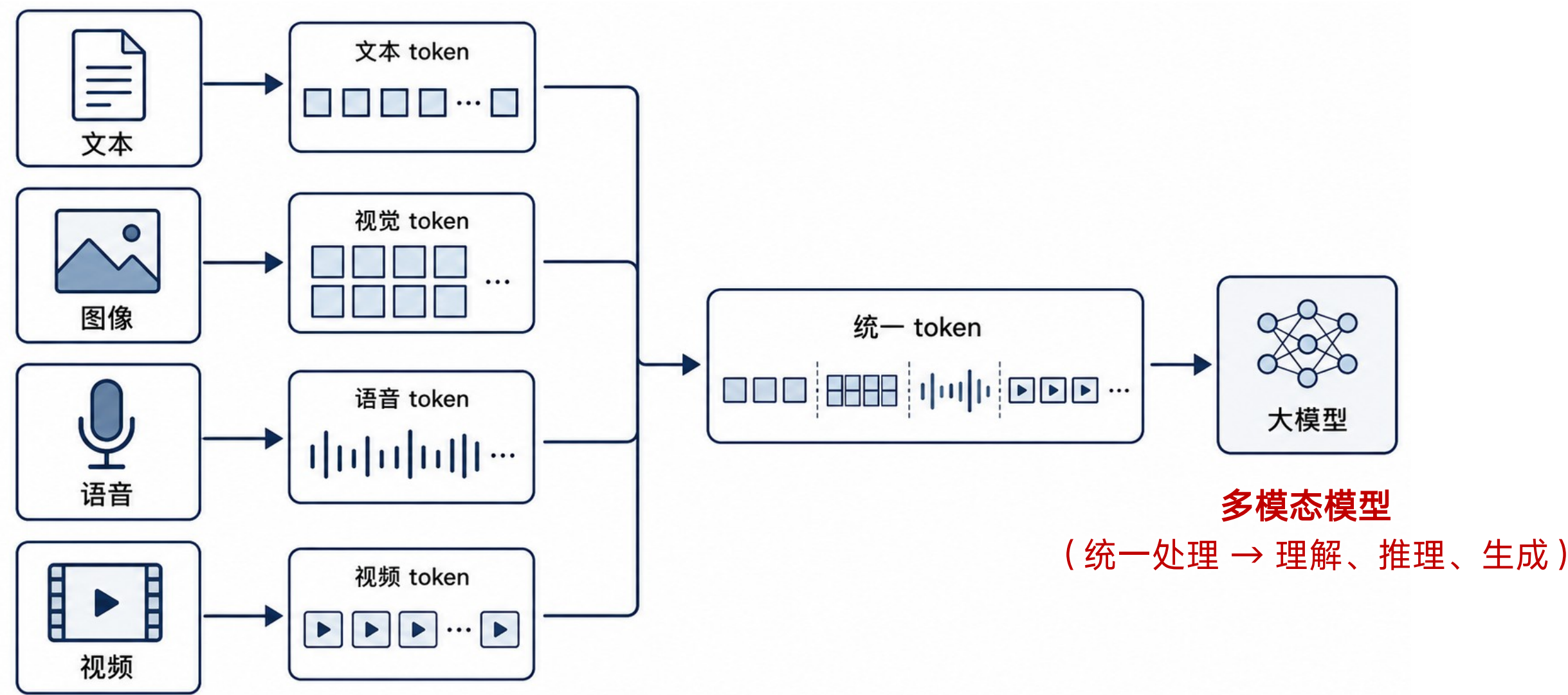
降低权重和带宽压力，提升推理速度

05 从语言模型到真实应用

多模态、具身智能和 recipes 工程实践入口

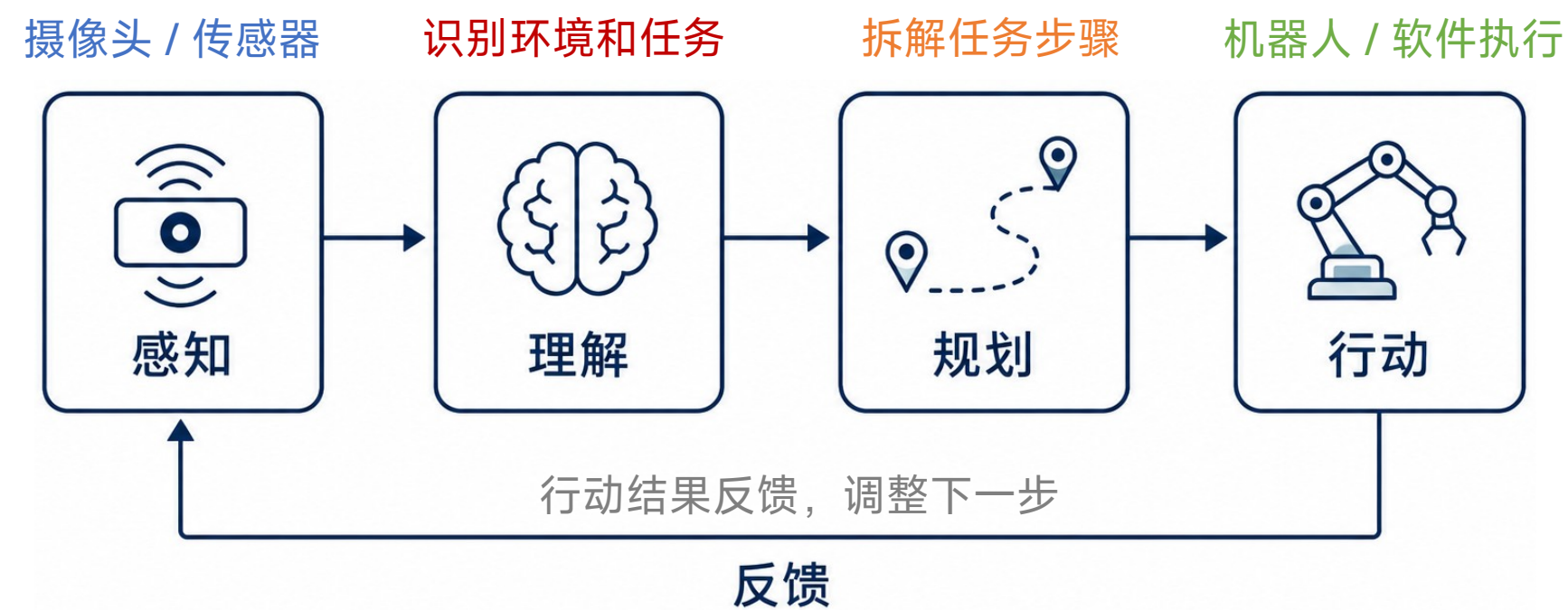
多模态：不同信号也可以变成模型输入

核心思路：不同信号先通过各自编码器转成 token 或向量，再交给大模型统一处理。



示例：输入一张图片 + "这张图里有什么风险？" → 图像变成视觉 token，文字变成文本 token → 模型结合两类信息生成回答

具身智能：感知、决策、行动形成闭环



推理能力会进入更多应用

 智能助手 长上下文、工具调用、记忆	 多模态生产力 文档、图像、视频协同	 行业智能化 金融、制造、医疗、教育	 端侧与机器人 更低延迟、更高可靠性
--	--	--	--

应用越接近真实用户，推理系统越需要兼顾**速度**、**成本**和**稳定性**。这些概念在 CANN recipes 仓库中都有对应的工程实现。

工程实践

这些概念会在 CANN recipes 仓里落到具体的工程对象：

模型结构

models/*/modeling_*.py → Tokenizer、Embedding、Attention、MLP

推理入口

infer.sh / README / YAML → Prefill/Decode 流程配置

性能分析

profiler、kernel_details、trace → TTFT/TPOT/TPS 观测

优化实践

融合算子、KV Cache 优化、量化、图模式

掌握这些基础概念后，再看实践课里的脚本、日志和 profiler 输出，就能明白它们在做什么了。理论指导实践，实践验证理论。

总结

将这四步连起来，就能开始阅读大模型推理工程



从文本到推理系统，从理论到工程实践。接下来，让我们在 CANN 上动手实践！

Thank you.

社区愿景：打造开放易用、技术领先的 AI 算力新生态

社区使命：使能开发者基于 CANN 社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的 CANN 生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上 CANN 社区获取干货



关注 CANN 公众号获取资讯